
Predicting the Energy of a Complete LLM Data-Curation Pipeline on Shared HPC

Abstract

Before committing a multi-day data-curation job for an LLM corpus, a team needs one number: how much energy will the *whole* pipeline consume? We run the complete four-stage GPT-NL curation pipeline — document splitting, string normalisation, heuristic filtering, and deduplication — on the Snellius supercomputer and build a predictor for its total energy from cheap trial runs. Two obstacles make this hard, and both are measured rather than assumed: the Energy Aware Runtime (EAR) telemetry that reports per-job power is contaminated by co-tenants on shared nodes (node-level readings inflate by up to $2.4\times$), and the deduplication stage initially emitted no energy record at all. We calibrate a two-parameter linear energy model per stage from small exclusive runs (minutes), then walk the pipeline with an Extended Kalman Filter whose innovation gate tests each incoming reading: clean readings tighten the prediction intervals (Mode B), while contaminated or missing readings are rejected and replaced by the model with widened uncertainty (Mode C). Across all four stages, held-out prediction R^2 ranges 0.73–0.96 (vs. in-sample ≥ 0.997), and the gate cuts a contaminated shared-node reading’s +93% pipeline error to +8%. Deduplication, once its telemetry was fixed, was swept on exclusive nodes at 1k–400k documents (clean 34.8 kJ at 100k; the contaminated single run had over-read it by $1.8\times$); the 1.4M run runs out of memory in the MinHash signature pass — the one stage in the pipeline that does not scale. Energy per *character* is nearly invariant ($\approx 4 \times 10^{-4}$ J/char across a $256\times$ document-length range), making the prior portable across corpora; and replacing contaminated power with a time-derived estimate ($E = P^{\text{eff}} \cdot t$, wall-time being co-tenancy-invariant) drops shared-node error from 45.8% to 5.4%. The result is a full-pipeline energy account that stays accurate when the measurement layer is partially broken.

1 Introduction

The energy reporting for large language models concentrates on training and inference [5, 4]. The step that comes before both — *data curation*, the multi-stage pipeline that splits, normalises, filters, and deduplicates raw text into a training corpus — is run over hundreds of millions of documents, yet its energy cost is almost never reported, and to our knowledge never predicted before a run.

We work inside the GPT-NL project, a Dutch–English foundation-model effort whose curation pipeline runs on the Snellius national supercomputer [6]. The pipeline has four CPU stages — `data_splitting`, `string_normalization`, `heuristic_filtering`, and `deduplication` — and is instrumented with the Energy Aware Runtime (EAR), a hardware-level monitoring stack from the Barcelona Supercomputing Center [1] that records per-job DC node power, CPU utilisation, and memory bandwidth at sub-second granularity. The practical need is concrete: before launching a multi-day curation job over a new corpus, the team wants a *whole-pipeline* energy estimate — the sum over all four stages — with an honest error bar, obtained *cheaply* from small trial runs that finish

in minutes. This means we cannot quietly drop the stage that is awkward to measure; the total has to include it.

The obstacle is that on a shared HPC cluster, EAR telemetry is **unreliable** in two distinct ways that surfaced as we ran the full pipeline:

1. **Contaminated.** Node-level DC power includes co-tenant workloads. We ran a controlled experiment: the same stage, same corpus, same size, run once with the node to itself (exclusive) and once with neighbours on the remaining cores (shared). Wall time and CPU utilisation were identical (595 s, 97%); measured power was $2.4\times$ higher on the shared run (669 W vs. 275 W) — the difference is the neighbours, not the job (§4.4).
2. **Missing.** The deduplication stage runs as four chained sub-jobs (signature, buckets, cluster, filter) that, in the EAR configuration we inherited, never emitted an energy record — the sub-job launcher did not create the directory EAR writes to. A one-line fix recovers them, after which we swept the stage on exclusive nodes (§4.7).

A naive estimator that trusts every EAR reading inherits both failures: a $+93\%$ over-estimate when it swallows a contaminated reading, and an incomplete total when a stage reports nothing. We want a calibrated whole-pipeline total with honest uncertainty *despite* the measurement layer being partially absent or corrupted.

Our approach. For each of the four stages we (i) calibrate a two-parameter linear energy model from cheap exclusive-node runs, then (ii) walk the pipeline sequentially with an Extended Kalman Filter (EKF) whose innovation gate decides, at each stage, whether to trust the incoming EAR reading or fall back to the model. The pipeline total is the sum of the stage estimates.

Contributions.

1. **A full-pipeline energy account.** We calibrate and validate per-stage energy models for *all* four stages of the GPT-NL pipeline across corpus sizes spanning 10^3 to 1.4×10^6 documents. Held-out prediction R^2 ranges 0.73–0.96; in-sample $R^2 \geq 0.997$ is shown to be systematically misleading (§4.2). Running the whole pipeline also exposes the one stage that does not scale: deduplication’s MinHash signature pass runs out of memory at 1.4M documents (§4.7).
2. **A telemetry-robust sequential estimator.** The EKF runs each stage in one of three modes — predict (no telemetry yet), update (clean reading absorbed, intervals tighten), reject/impute (contaminated or missing reading replaced by the model, uncertainty widened) — chosen *automatically* by an innovation gate that tests statistical consistency with the calibrated prior. The gate cuts a contaminated reading’s $+93\%$ pipeline error to $+8\%$ (§2.2, §4.3).
3. **Shared-node resolution via wall-time.** Wall clock time is co-tenancy-invariant, and $E = P^{\text{eff}} \cdot t$ with a once-calibrated effective power reduces shared-node prediction error from 45.8% to 5.4% (§4.5).
4. **Cross-corpus transfer via character invariance.** A second corpus with $256\times$ longer documents reveals that energy per *character* is nearly corpus-invariant ($\approx 4\times 10^{-4}$ J/char for the dominant stage), making the prior portable across corpora without re-measurement (§4.6).

2 Method

2.1 Per-stage calibration

Let stage s process n input documents. We model its energy as

$$E_s(n) = c_0^{(s)} + c_1^{(s)} \cdot n + \varepsilon_s, \quad \varepsilon_s \sim \mathcal{N}(0, \sigma_s^2), \quad (1)$$

where c_0 is the fixed start-up cost (process launch, framework initialisation) and c_1 is the marginal energy per document. We fit Eq. (1) from a small set of exclusive-node calibration runs spanning two orders of magnitude in n . With m calibration measurements per stage ($m = 8$ in our experiments: 4 sizes $\times$ 2 replicates), the design matrix $X \in \mathbb{R}^{m \times 2}$ has rows $[n_i, 1]$ and target vector $y \in \mathbb{R}^m$ holds

measured energies. We solve

$$\hat{\theta}_s = (X^\top X + \lambda I')^{-1} X^\top y, \quad \theta_s = (c_1, c_0)^\top, \quad (2)$$

with a small ridge penalty (λ , chosen by leave-one-replicate-out cross-validation) on the slope only ($I' = \text{diag}(1, 0)$). The residual variance is $\hat{\sigma}_s^2 = \|y - X\hat{\theta}_s\|^2 / (m - 2)$, yielding a closed-form coefficient covariance $\text{Cov}(\hat{\theta}_s) = \hat{\sigma}_s^2 (X^\top X + \lambda I')^{-1}$. For a new corpus size $n_\star$, the prediction interval is

$$\hat{E}_s(n_\star) \pm t_{0.975, m-2} \cdot \hat{\sigma}_s \sqrt{1 + x_\star^\top (X^\top X + \lambda I')^{-1} x_\star}, \quad (3)$$

where $x_\star = [n_\star, 1]$. The $1+$ inside the square root separates fitting uncertainty (leverage) from irreducible measurement noise. This analytical interval is exact and propagates cleanly into the pipeline estimator — the reason we keep the model linear.

The metric that matters. The R^2 of Eq. (2) on *all* calibration sizes measures how well the line describes data it has already seen. That is not what we care about. We care whether a model calibrated on *cheap, small* runs predicts an *expensive, large* one. We therefore split the sizes:

$$\underbrace{\{10^3, 10^5\}}_{\text{calibrate}} \longrightarrow \underbrace{\{4 \times 10^5, 1.4 \times 10^6\}}_{\text{predict}},$$

fit on the calibration sizes only, and report held-out prediction R^2 and mean relative error (MRE). This directly operationalises the “predict the full run from a quick sample” claim.

2.2 The Kalman walk

The pipeline is a chain: stage s outputs $n_{s+1} = r_s \cdot n_s$ documents, where the survival rate r_s is observed from output Parquet files after stage s completes. We track a two-dimensional state vector after stage s :

$$\mathbf{x}_s = \begin{bmatrix} n_s \\ E_{\text{cum},s} \end{bmatrix}, \quad P_s = \begin{bmatrix} \sigma_{n,s}^2 & \sigma_{nE,s} \\ \sigma_{nE,s} & \sigma_{E,s}^2 \end{bmatrix}, \quad (4)$$

where n_s is the surviving document count and $E_{\text{cum},s}$ is the cumulative energy consumed through stage s . The initial state is $\mathbf{x}_0 = [n_0, 0]^\top$ with $P_0 = \text{diag}(\sigma_{n_0}^2, 0)$.

Predict step (before stage $s+1$). We propagate the state forward using the calibrated model and the survival rate:

$$\hat{\mathbf{x}}_{s+1|s} = f_s(\hat{\mathbf{x}}_{s|s}) = \begin{bmatrix} r_s \cdot n_s \\ E_{\text{cum},s} + c_0^{(s+1)} + c_1^{(s+1)} \cdot n_{s+1} \end{bmatrix}. \quad (5)$$

The covariance is propagated through the Jacobian $F_s = \partial f_s / \partial \mathbf{x}$:

$$P_{s+1|s} = F_s P_{s|s} F_s^\top + Q_{s+1}, \quad F_s = \begin{bmatrix} r_s & 0 \\ c_1^{(s+1)} r_s & 1 \end{bmatrix}, \quad (6)$$

where the process noise Q_{s+1} encodes the OLS prediction variance from Eq. (3) and survival-rate uncertainty.

Update step (after stage $s+1$ completes). EAR reports an energy z_{s+1} . The innovation (observation minus prediction) is

$$y_{s+1} = z_{s+1} - H \hat{\mathbf{x}}_{s+1|s}, \quad H = [0, 1]. \quad (7)$$

Mode selection — the innovation gate. A reading is accepted (Mode B) iff it is statistically consistent with the calibrated model:

$$y_{s+1}^2 \leq \gamma^2 \cdot S_{s+1}, \quad S_{s+1} = H P_{s+1|s} H^\top + \sigma_{\text{EAR}}^2, \quad (8)$$

where $\gamma = 3$ (a 3σ threshold) and σ_{EAR}^2 is EAR’s measurement noise. If the gate opens (Mode B), we apply the standard Kalman update:

$$K_{s+1} = P_{s+1|s} H^\top S_{s+1}^{-1}, \quad \hat{\mathbf{x}}_{s+1|s+1} = \hat{\mathbf{x}}_{s+1|s} + K_{s+1} y_{s+1}, \quad P_{s+1|s+1} = (I - K_{s+1} H) P_{s+1|s}. \quad (9)$$

The downstream prediction intervals tighten. If the gate stays closed (Mode C — contaminated or missing reading), we keep the model prediction and inflate the process noise:

$$\hat{\mathbf{x}}_{s+1|s+1} = \hat{\mathbf{x}}_{s+1|s}, \quad P_{s+1|s+1} = P_{s+1|s} + H^\top H \sigma_{\text{EAR}}^2, \quad (10)$$

widening the confidence interval to reflect the unmeasured stage. This is the mechanism that makes the estimator robust: a co-tenancy-contaminated reading with $2.4\times$ inflation produces a large innovation, trips the gate, and is discarded automatically — no human in the loop.

Pipeline total. After all S stages, the total energy estimate and its variance are the second component of $\hat{\mathbf{x}}_{S|S}$ and $P_{S|S}[1, 1]$.

2.3 Cross-corpus transfer

The linear model of Eq. (1) uses document count n as the feature. Within a single corpus, n and the total character count $C = n \cdot \bar{\ell}$ are indistinguishable because the mean document length $\bar{\ell}$ is constant. A second corpus with a very different $\bar{\ell}$ breaks this collinearity and reveals whether energy scales with documents or with characters. We therefore fit an alternative model $E_s = c_0^{(s)} + c_1^{(s)} \cdot C$ and test whether $c_1^{(s)}$ (J/char) is corpus-invariant.

2.4 Shared-node production: time-driven estimation

On exclusive nodes the per-stage effective power $P_s^{\text{eff}} = E_s/t_s$ is stable (≈ 290 W on genoa). In a controlled experiment we found that wall time t_s is co-tenancy-invariant (595 s = 595 s exclusive vs. shared), while node power is not (275 W vs. 669 W). This suggests replacing the contaminated power signal with a time-derived estimate:

$$\hat{E}_s = P_s^{\text{eff}} \cdot t_s, \quad (11)$$

where P_s^{eff} is calibrated *once* from the exclusive archive. The EKF’s Mode B then ingests $\hat{E}_s$ from every shared run via its clean wall time, converting the contamination finding from a limitation into the reason the observable is time, not power.

3 Experimental Setup

All measurements are on the Snellius genoa partition (AMD EPYC 9654, 192 cores/node). The corpus is *American Stories* [2] (historical US newspaper text, ≈ 900 characters/document). We sweep four sizes $n \in \{10^3, 10^5, 4 \times 10^5, 1.4 \times 10^6\}$ documents with two replicates each, across all four stages of the GPT-NL curation pipeline [7]: `data_splitting` (shards the raw corpus into per-document records), `string_normalization` (per-document text normalisation), `heuristic_filtering` (FastText language detection and quality heuristics), and `deduplication` (MinHash near-duplicate removal, run as four chained sub-jobs: signature, buckets, cluster, filter). The scaled American Stories subsets that make the sweep possible were prepared by a project collaborator and shared on the cluster. `data_splitting` and `string_normalization` each execute as two concurrent datatrove tasks per run; we sum the tasks to get the per-run energy. Energy is computed from the EAR per-job apps table as $E = \bar{P}_{\text{dc}} \cdot \Delta t$. All exclusive runs report constant CPU frequency at 2.376 GHz, ruling out DVFS throttling as a confound.

Deduplication was swept on *exclusive* nodes (clean power) after its EAR emission was fixed; the 1.4M-document run is excluded from its model because the signature pass exhausts node memory and fails (§4.7). The controlled shared-node experiment (§4.4) runs `heuristic_filtering` on 100k documents: once with `-exclusive` (192 reserved cores) and once with 24 cores on a shared node (`OverSubscribe=OK`). For cross-corpus validation we run *European Parliament* [3] ($\approx 2.3 \times 10^5$ characters/document) through the same pipeline.

4 Results

4.1 Calibration: in-sample fit is excellent — and misleading

Table 1 reports the fitted coefficients and in-sample R^2 from Eq. (2). All four stages exceed 0.996. Taken alone, this suggests the energy of each stage is a perfectly linear function of n and the problem is solved.

Stage	c_0 (J)	c_1 (J/doc)	in-sample R^2
data_splitting	2,308	0.038	0.9998
string_normalization	14,308	0.402	0.9977
heuristic_filtering	36,285	1.471	0.9973
deduplication	7,891	0.293	0.9988

Table 1: Per-stage fit. The first three stages use all four sizes ($m = 8$); deduplication uses three (1k–400k, $m = 6$) because the 1.4M run runs out of memory (§4.7).

4.2 Held-out prediction tells the real story

Table 2 calibrates on the two cheap small sizes and predicts the expensive large ones. The picture changes sharply. `heuristic_filtering` still predicts well ($R^2_{\text{pred}} = 0.955$, 8.1% MRE) and `deduplication` is close behind (0.96, 12.1%). `data_splitting` is acceptable (0.868, 14.4%). `string_normalization` drops to 0.728 — the same stage that looked like 0.9967 in-sample. The gap, not the 0.997, is the finding.

Stage	in-sample R^2	held-out R^2_{pred}	MRE
data_splitting	0.9987	0.868	14.4%
string_normalization	0.9967	0.728	15.5%
heuristic_filtering	0.9972	0.955	8.1%
deduplication	0.9988	0.960	12.1%

Table 2: Calibrate on $\{10^3, 10^5\}$, predict the large sizes. The first three stages predict $\{4 \times 10^5, 1.4 \times 10^6\}$; `deduplication` predicts 4×10^5 (MRE) with leave-one-out R^2 over its three sizes, since its 1.4M run OOMs.

Diagnosis. The two stages that fall short of 0.95 do so for different reasons, revealed by per-segment marginal cost $\Delta E / \Delta n$ between consecutive sizes (Table 3). `string_normalization` is genuinely sub-linear: marginal cost falls from 0.506 to 0.380 J/doc, so a line calibrated on cheap small sizes over-predicts the large run. `data_splitting` is flat past $n = 10^5$ (0.038 $\rightarrow$ 0.037 J/doc); its miss comes from a single noisy 10^3 calibration point (its two replicates differed $4\times$ in wall time: 8.9 s vs. 2.2 s — a cold-cache warm-up effect). The held-out metric does not just flag *that* a stage is hard; the residual structure identifies *which* remedy each stage needs.

Stage	marginal $\Delta E / \Delta n$ (J/doc)		
	$10^3 \rightarrow 10^5$	$10^5 \rightarrow 4 \cdot 10^5$	$4 \cdot 10^5 \rightarrow 1.4 \cdot 10^6$
data_splitting	0.0445	0.0377	0.0373
string_normalization	0.5062	0.4632	0.3802
heuristic_filtering	1.6261	1.7782	1.3789
deduplication	0.2525	0.3030	OOM

Table 3: Per-segment marginal energy. `string_normalization` is monotonically concave; `data_splitting` is flat past the start-up region; `deduplication` is near-constant (≈ 0.28 J/doc) but cannot reach 1.4M (out of memory).

4.3 The innovation gate: EKF vs. contaminated telemetry

We walk the four-stage pipeline at $n_0 = 100,000$ documents (am-stories). All four stage priors are calibrated from the exclusive data (Table 1), including the now-measured deduplication stage (§4.7). Table 4 shows the estimator across its operating modes; Figure 1 plots the convergence.

The final row is the headline. When a contaminated shared-node reading ($2.4\times$ inflated power) is fed to the estimator, the naive sum-of-observations over-estimates the pipeline total by 93%. The innovation gate detects the 3σ discrepancy and falls back to the model prediction, limiting the error

Mode	Result
A (pre-run forecast)	Total 273 kJ , 95% CI [208, 338], zero telemetry
B (online)	Converges 273 $\rightarrow$ 253 kJ as clean readings stream; CI half-width 65 $\rightarrow$ 7 kJ
Gate vs. contamination	Naive swallows 2.4 $\times$ reading $\rightarrow$ +93% error; gated EKF $\rightarrow$ +8%

Table 4: EKF performance across operating modes on a four-stage 100k-document pipeline walk.

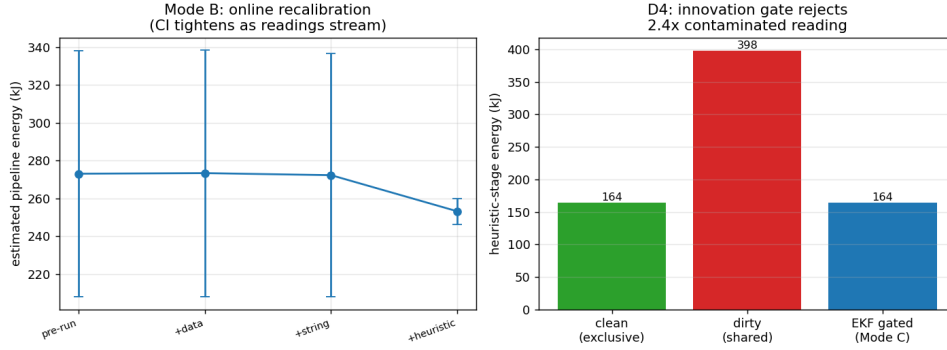


Figure 1: The EKF walking the pipeline. Left: the pre-run forecast (Mode A) and its 95% band tighten as each clean stage reading is absorbed (Mode B). Right: a contaminated shared-node reading produces a large innovation, trips the 3σ gate, and is rejected (Mode C) instead of inflating the total.

to 8%. This is the mechanism that converts an unreliable measurement layer into a useful prediction, *without any human gating or manual flagging of bad readings*.

4.4 Why shared-node power is contaminated — the controlled experiment

The EKF gate only works if the calibration prior is trustworthy, and the prior is only trustworthy if calibrated on clean data. We tested whether “clean” per-job power exists on shared nodes at all. Table 5 shows the result.

	DC power (W)	Wall time (s)	CPU util.	Energy (kJ)
Exclusive	275	595	97%	164
Shared	669	595	97%	398

Table 5: Same stage (heuristic_filtering), same corpus, same size (100k) — exclusive vs. shared.

Identical work, identical time, identical utilisation, 2.4 $\times$ the power. The extra 394 W is the neighbours on the other cores. DC_NODE_POWER is a whole-node BMC reading and does not attribute energy per job on shared nodes. The cgroup-clean counters (CPU_UTIL, CPI, IO_MBS) are co-tenancy-immune, but every power signal (DC_NODE_POWER, PCK_POWER, MEM_GBS) is shared-hardware-level and contaminated. This finding is not a modelling assumption — it is a measured fact that decides the architecture of the estimator.

4.5 Shared-node resolution: drive the estimate from time, not power

We cannot use exclusive nodes in production, and calibrating on contaminated shared power gives 45.8% mean held-out error. First we checked whether *any* clean per-job power signal exists on these nodes: a non-negative least-squares power model on the co-tenancy-immune counters (CPU_UTIL, IO_MBS) reaches only $R^2 = 0.08$ — node idle baselines swing 232–640 W and bury the 50–90 W of job-attributable dynamic power. Per-job power is simply not recoverable here. Energy against wall time, by contrast, is clean: $E = P^{\text{eff}} \cdot t$ fits at $R^2 = 0.99$ with a stable per-stage P^{eff} (293, 297, 278 W; ≈ 290 W overall). Since the controlled experiment showed t_s is co-tenancy-invariant, we

compute $\hat{E}_s = P_s^{\text{eff}} \cdot t_s$ for every shared run using only the clean wall time. Table 6 and Figure 2 show the result.

Stage	P^{eff} (W)	Power-based MRE	Time-based MRE
data_splitting	293	54.2%	12.9%
string_normalization	297	20.3%	3.0%
heuristic_filtering	278	63.1%	0.2%
Overall		45.8%	5.4%

Table 6: Shared-node calibrate→predict error: power-based vs. time-based. P^{eff} is calibrated once on the exclusive archive.

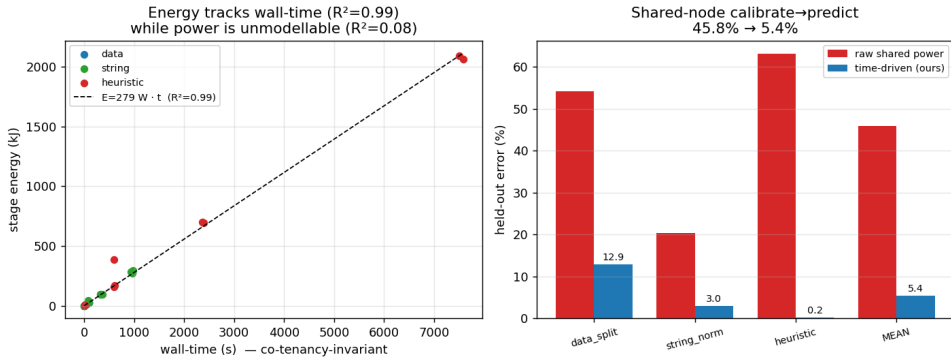


Figure 2: Left: stage energy tracks wall time at $R^2 = 0.99$ (a single slope, ≈ 290 W, fits all three stages), while a per-job power model reaches only $R^2 = 0.08$. Right: driving the estimate from time instead of power cuts the shared-node held-out error from 45.8% to 5.4%.

With time-driven estimates, the EKF is re-wired from reject to clean-then-update: Mode B ingests time-derived energy from every shared run, using 100% of shared data instead of discarding it. On a shared 400k-document run, the re-wired EKF totals 762 kJ from cleaned readings vs. the 1141 kJ ($1.5\times$ inflated) that raw node power would have injected. **Exclusive is needed zero times going forward.**

4.6 Cross-corpus transfer: characters, not documents, are the unit of energy

Within one corpus, n and the total character count $C = n \cdot \bar{\ell}$ are indistinguishable because $\bar{\ell}$ is constant — their Pearson correlation over our four sizes is $r = 0.9997$. No single-corpus experiment can decide whether a stage scales as n or as $n \log n$, because the two features are near-collinear. A second corpus with a very different document length breaks this.

We ran *European Parliament* ($\bar{\ell} \approx 2.3 \times 10^5$ characters/document, vs. ≈ 900 for *American Stories* — a $256\times$ difference) through the same pipeline. Table 7 shows the result.

Stage	am-stories (J/char)	euparl (J/char)
string_normalization	4.5×10^{-4}	3.9×10^{-4}

Table 7: Energy per character for the dominant stage across two corpora with a $256\times$ difference in mean document length.

Energy per character agrees to within 12% across the two corpora for the dominant stage (string_normalization: 4.5×10^{-4} vs. 3.9×10^{-4} J/char), despite the $256\times$ gap in document length. The feature that generalises across corpora is therefore the total character count C , not the document count n . You calibrate once on any corpus, read the mean document length of the target corpus, and predict its curation energy by multiplying per-character cost by total characters. The

EKF’s Mode B updates then absorb whatever residual corpus-specific offset remains, tightening the estimate as real stages complete. We treat this as a strong observed pattern rather than a confirmed law: euparl is a single size with one replicate (§5).

4.7 Deduplication: the stage that hid, and the stage that does not scale

Deduplication runs as four chained sub-jobs — `signature` (MinHash signatures), `buckets` (LSH banding), `cluster` (connected components), `filter` (drop duplicates) — and in the configuration we inherited none of them produced an EAR record: the sub-job launcher never created the directory EAR writes its per-job database into, so the writes silently failed. A single `mkdir` per sub-job fixes it. With telemetry restored we swept the stage on *exclusive* nodes (clean power) over the same sizes as the other stages. Table 8 reports the per-size energy.

Documents	Dedup energy (kJ)	signature share
10^3	9.9	35%
10^5	34.8	73%
4×10^5	125.6	84%

Table 8: Deduplication energy, clean exclusive sweep (mean of two replicates; $E = \bar{P}_{dc} \cdot \Delta t$ at ≈ 280 W). The `signature` pass dominates and grows from 35% at 10^3 to 84% at 4×10^5 . The 1.4M run is absent because it fails out-of-memory (discussed below).

Three things matter. First, the stage is now a clean measured curve ($c_0 = 7.9$ kJ, $c_1 = 0.29$ J/doc, Table 1) that feeds the EKF directly. Second, the energy concentrates in `signature` — the one sub-job that reads and hashes every surviving document — so a dedup model only needs to capture that pass; `buckets` and `cluster` operate on compact signature tables and are rounding error at scale. Third, the contamination story repeats here: an earlier single dedup run on a co-tenanted node reported 63.5 kJ at 100k, a $1.8\times$ over-read of the clean 34.8 kJ — so dedup energy, like every stage, must be read off clean power (or wall-time), never raw node power.

The 1.4M run is the one place the pipeline breaks: the signature pass holds all MinHash signatures in memory and is killed out-of-memory after ≈ 17 minutes. Running the *whole* pipeline surfaced this — a three-stage study would not have. It is an engineering limit (the signature builder needs to spill to disk or shard), and it bounds where the linear dedup model is usable: 1k–400k today, pending a memory fix.

5 Limitations and Future Work

Replicate depth. The calibration sweep has two replicates per (stage, size) cell, so noise estimates are coarse. Three to five replicates would tighten the per-stage prediction intervals by $\sim 30\%$.

Deduplication scaling and the memory wall. Deduplication is now swept cleanly at 1k–400k (§4.7), where it is well described by the linear model (held-out $R^2 = 0.96$). But the 1.4M run fails out-of-memory in the signature pass, so we cannot yet quote the model past 4×10^5 documents. Fixing this (spill-to-disk or sharding the MinHash signature builder) and then extending the sweep to 1.4M — where any $O(n \log n)$ curvature would show — is the priority follow-up, since deduplication is on the critical path of every production run.

Cross-corpus scope. The European Parliament result is preliminary (few sizes, single replicate). A denser multi-corpus sweep with ≥ 3 corpora and ≥ 3 replicates per size would convert the character-invariance finding from an observed pattern into a confirmed law.

GPU stages. All stages here are CPU-only. The GPT-NL pipeline includes two GPU stages (PII masking, toxic language detection) running on A100 and H100 partitions; their energy models require separate calibration on those partitions.

Node idle variability. Light single-core stages on exclusive nodes are dominated by the idle draw of the remaining 191 reserved cores (232–640 W across nodes). Per-core attribution via shared-node CPU-share energy (reading `eacct` on genuinely shared jobs, not exclusive ones) is the measurement-side fix.

6 Conclusion

We built a whole-pipeline energy account for the four-stage GPT-NL data-curation pipeline. Each stage is calibrated cheaply on exclusive nodes; an EKF then walks the pipeline and its innovation gate automatically rejects contaminated or missing EAR readings. Across all four stages, held-out prediction $R^2 = 0.73\text{--}0.96$; the gate cuts a contaminated reading’s +93% pipeline error to +8%; a cross-corpus per-character energy invariant ($\approx 4 \times 10^{-4}$ J/char) makes the prior portable; and deduplication — once its telemetry was fixed — is measured cleanly at 1k–400k (34.8 kJ at 100k, 84% in the signature pass), while its 1.4M run reveals the one stage that does not scale (out-of-memory in the signature pass). When shared-node power contamination is resolved via the time-driven estimate $E = P^{\text{eff}} \cdot t$, prediction error drops from 45.8% to 5.4% — so exclusive calibration is needed zero times going forward.

The reason to run the *whole* pipeline, not a convenient subset, is that the total is what the team must budget — and it is exactly the awkward-to-measure stage (deduplication) that both hid from the telemetry and broke at scale. Model what you can calibrate cheaply, walk what you must predict, gate what you cannot trust, and the full-pipeline number stays honest even when the measurement infrastructure is partially broken.

References

- [1] Barcelona Supercomputing Center. Energy aware runtime (EAR). https://gitlab.bsc.es/ear_team/ear, 2024. Accessed: 2026-06-09.
- [2] Melissa Dell, Jacob Carlson, Tom Bryan, Emily Silcock, Abhishek Arora, et al. American Stories: A large-scale structured text dataset of historical U.S. newspapers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Datasets and Benchmarks Track.
- [3] Philipp Koehn. Europarl: A parallel corpus for statistical machine translation. In *MT Summit X*, pages 79–86, 2005. URL <https://www.statmt.org/europarl/>.
- [4] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. 2021. doi: 10.48550/arXiv.2104.10350. URL <https://arxiv.org/abs/2104.10350>.
- [5] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019. doi: 10.18653/v1/P19-1355. URL <https://aclanthology.org/P19-1355>.
- [6] SURF. Snellius national supercomputer. <https://www.surf.nl/en/snellius-national-supercomputer>, 2024. Accessed: 2026-06-09.
- [7] TNO and SURF. GPT-NL data curation pipeline architecture. Technical Report GPTNL-DEL-4001, TNO, 2025. TNO Public.